Group Project Documentation

MICROSERVICES WITH SPRING BOOT

KABAN PROJECT

Prepared by Group 2

Vũ Hoàng Phúc

Ngô Văn Bảo

Hà Vũ Long

Phạm Đinh Đạt

Nguyễn Quang Quyền

Hanoi City, 2026

Table of Contents

Revision History **4**
Executive Summary **5**
0.1. How to Read This Report 5
01 Project Requirements and Scope **6**
1.1. Problem Statement and Requirement Groups 6
1.2. Phase 1 Scope and Exclusions 6
1.3. Functional and Non-Functional Requirements 7
02 Architecture Design and Service Overview **8**
2.1. Logical Deployment Topology 8
2.2. Service Responsibilities 9
2.3. Technologies and Data Model 9
2.4. Authorization, Trust Boundaries, and Controls 9
03 Communication Diagrams — Two Main Use Cases **10**
3.1. Use Case 1 — Login and Protected API Request 10
3.2. Use Case 2 — Assign Task → RabbitMQ → Notification 11
3.3. Supporting Flow — Invitation and Membership 12
04 API Design **13**
4.1. API Conventions and Routing 13
4.2. Endpoint Matrix by Controller 13
4.3. Basic API Contract Examples 14
4.4. HTTP Status Codes and Authentication 15
05 Database Design **16**
5.1. Database per Service and Entity–Relationship Overview 16
5.2. Service-Owned Database Tables 16
5.3. Index and Transaction Design 17
5.4. Data Synchronization and Consistency 17
06 Main Demo Flow — Integrated Backend **18**
6.1. End-to-End Business Flow Diagram 18
Appendix A — Source Code Map by Topic **20**
Appendix B — Integration Checklist and Acceptance Criteria **21**
Scope of Interpretation 21

LIST OF TABLES

Table 0.1.1. How to read the main processing flows 5
Table 1.1.1. Business requirement groups 6
Table 1.3.1. Functional and non-functional requirements 7
Table 2.2.1. Responsibilities and dependencies of system services 9
Table 3.1.1. Login and Gateway authentication steps 10
Table 3.2.1. Processing phases for task assignment and notification creation 11
Table 4.1.1. API Gateway routing groups 13
Table 4.2.1. API endpoints grouped by service 13
Table 4.4.1. HTTP status codes and meanings 15
Table 5.2.1. Database tables and constraints by owning service 16
Table 6.1.1. Stages of the main demo scenario 19
Table A.1.1. Source code references grouped by feature 20
Table B.1.1. Integration checklist and acceptance criteria 21

LIST OF FIGURES

Figure 2.1.1. Vertical system architecture: HTTP enters through the Gateway and reaches four independently owned services and databases; Redis and RabbitMQ provide shared infrastructure. 8
Figure 3.1.1. Communication sequence for public login and the protected GET /users/me request. 10
Figure 3.2.1. Task assignment with two Accounts: Account 1 assigns the Task; RabbitMQ distributes an event between services; Account 2 requests its notification through the Gateway. 11
Figure 5.1.1. Logical data model. Database columns named card belong to the Task domain; there are no cross-database foreign keys between service-owned databases. 16
Figure 6.1.1. Chronological demo flow distinguishing Account 1 and Account 2, HTTP calls through the Gateway, and backend events distributed through RabbitMQ 18

Revision History

Editor	Date	Reason for Changes	Version

Executive Summary

Dev-Platform is a project-based task management platform. Each Project has an Owner and Members, and contains Boards → Columns → Tasks → Comments. Phase 1 includes an API Gateway, four backend services, four Microsoft SQL Server databases, Redis, RabbitMQ, a React web demo, and a Flutter mobile application.

In the integrated operational flow, users interact through React or Flutter. The Gateway validates JWTs, retrieves Account authorization snapshots from Redis, and routes requests. Each service writes to its own SQL Server database. When a Task is assigned or a Comment mentions a member, Task Service writes an Outbox event. RabbitMQ delivers the event to Notification Service, which records the Inbox event and Notification. Users then retrieve notifications through an API.

Purpose: This report supports architecture study, academic presentations, team development planning, and preparation for a live demonstration of the main business flow.

Note: Do not include real .env credentials, passwords, or private keys in the report. Example requests use illustrative data and placeholders.

0.1. How to Read This Report

Table 0.1.1. How to read the main processing flows

Flow	Description
Business flow	Sequence from Project creation to Task collaboration and receiving Notifications.
Request flow	HTTP and WebSocket routing through the Gateway, service authorization, and the owning database.
Event flow	Task Outbox → RabbitMQ → Notification Inbox → Notification Database.
Demo flow	Backend mode setup, request endpoints, and points where results are verified.

01 Project Requirements and Scope

1.1. Problem Statement and Requirement Groups

Table 1.1.1. Business requirement groups

Requirement Group	Business Description	Primary Actors
Account & Authentication	Admin creates and manages Accounts; business users sign in, receive JWTs, and must be ACTIVE. No public self-registration.	System Admin, User
Project & Membership	Create a Project with its default Owner; invite existing Accounts; accept or reject Invitations; manage Members and transfer ownership.	Owner, Member
Kanban Workspace	Manage Boards within Projects; order Columns; place Tasks in Columns; drag or move Tasks.	Owner, Member
Collaboration	Assign Tasks to Members; post Comments and mention other Project Members.	Owner, Member
Notification	Receive task-assignment and mention notifications; view personal notifications and mark them as read.	Member
Realtime & UX	Board WebSocket uses a one-time ticket; the frontend loads additional Tasks by Column and refreshes the UI.	Web User
Reliability & Security	Project-based role-based access control, Redis authorization snapshots, Outbox/Inbox, consumer deduplication, retries, and dead-letter queues.	System, Operator

1.2. Phase 1 Scope and Exclusions

- In scope for Phase 1: Account/Login, Project/Member/Invitation, Board/Column, Task/Comment, Notification, Gateway, RabbitMQ/Redis integration foundations, React web demo, and Flutter mobile skeleton.
- Out of scope for Phase 1: automatic Account deletion and Project cascading, automatic Owner succession, and detailed Git Core, Pull Request, and Workflow capabilities planned for Phase 2.
- Business rules: System Admin manages Accounts but does not automatically obtain Project permissions. The Owner manages Projects and Boards; Members collaborate on Tasks, Columns, and Comments. A PENDING Invitation does not grant Project Membership.

1.3. Functional and Non-Functional Requirements

Table 1.3.1. Functional and non-functional requirements

Functional Requirements	Non-Functional / Technical Requirements
Login, Account directory, system role management	RSA256 JWT verification, signature and expiry; separate Admin authority from Project permissions.
Create Project → Board → Column → Task	Service-owned data; no foreign keys spanning databases.
Invite users, accept invitations, change Members	Snapshot versions and fail-closed behavior when trusted authorization data is missing.
Assign Task, mention Member, view Notification inbox	Eventual consistency; Outbox/Inbox prevent lost or duplicate processing.
Board realtime updates, per-Column pagination	WebSocket ticket time-to-live and keyset cursor by position DESC and id DESC.

02 Architecture Design and Service Overview

2.1. Logical Deployment Topology

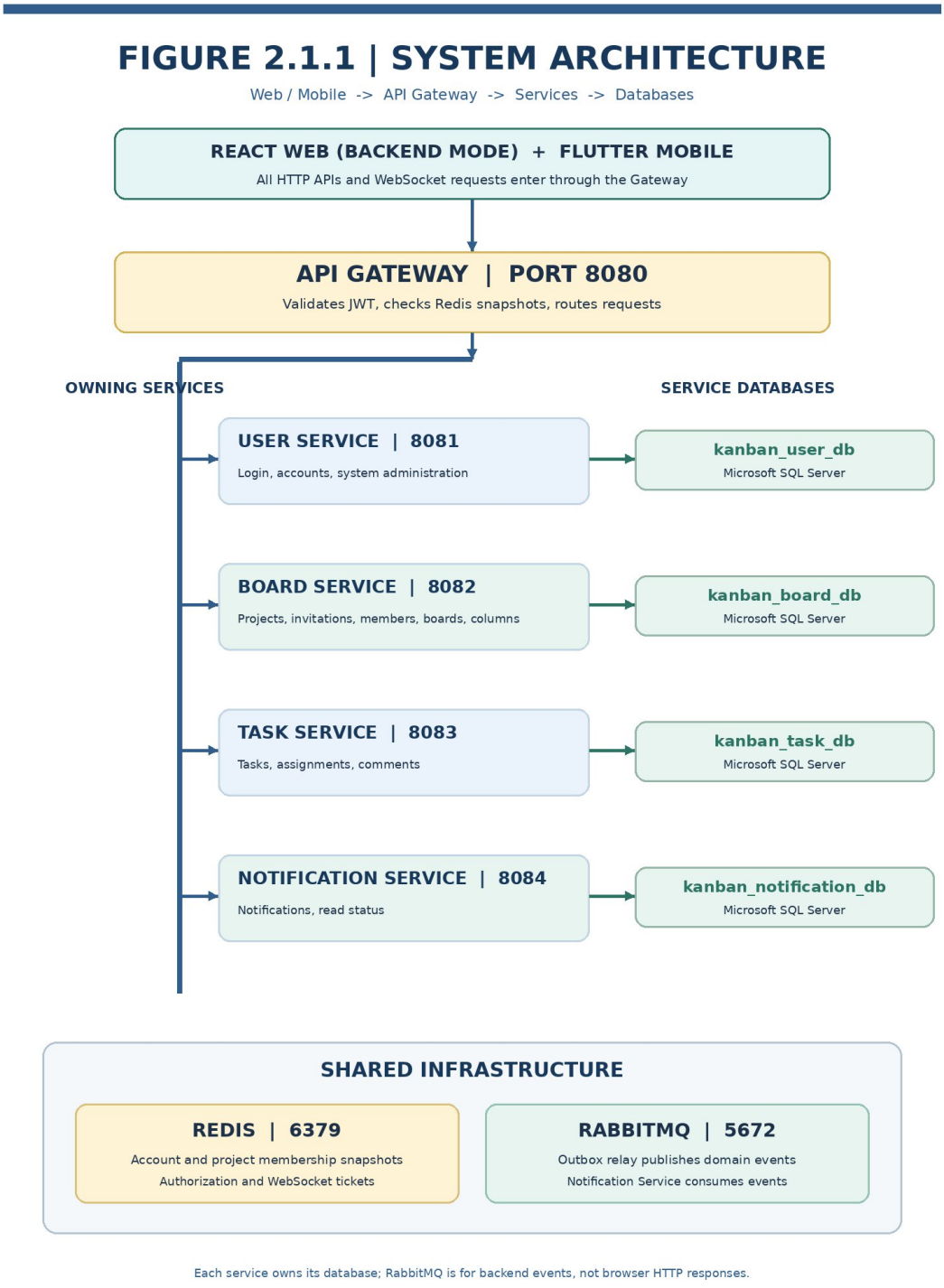


Figure 2.1.1. Vertical system architecture: HTTP enters through the Gateway and reaches four independently owned services and databases; Redis and RabbitMQ provide shared infrastructure.

In the integrated deployment, clients connect to the API Gateway. The Gateway forwards each request to the correct backend, while each service validates its business rules and owns persistence. SQL Server contains four separate databases; Redis supports authorization snapshots and WebSocket tickets; RabbitMQ handles asynchronous business events. Spring Boot applications run independently. Local infrastructure is initialized with Docker Compose and SQL Server.

2.2. Service Responsibilities

Table 2.2.1. Responsibilities and dependencies of system services

Service / Port	Core Responsibilities	Dependencies and Owned Resources
API Gateway :8080	GatewayAuthenticationFilter, routing, and WebSocket tickets. Authenticates Accounts and forwards identity but does not decide Project roles.	RSA256 public key from User Service; Redis auth:account:*; HTTP and /ws proxy.
user-service :8081	Login, JWT issuance, current user, directory/profile, and Admin account creation, status, and roles.	kanban_user_db; Account snapshot synchronization to Redis; Outbox.
board-service :8082	Project, Membership, Invitation, Board, and Column. There is no separate project-service.	kanban_board_db; Project membership snapshots; internal Board and Membership lookups.
task-service :8083	Task management, assignments, Comments, keyset cursor, WebSocket events, and assignment cleanup.	kanban_task_db; Redis Account/Member snapshots; Board internal REST; Task Outbox.
notification-service :8084	Consume card.assigned and comment.mentioned; list Notifications and mark as read.	kanban_notification_db; RabbitMQ consumer and Inbox deduplication.

2.3. Technologies and Data Model

- Backend: Java 21, Spring Boot, Spring Cloud Gateway (WebFlux), Spring Security/REST, and Spring Data JPA; Maven multi-module build with shared event-contracts and messaging-support modules.
- Storage: Microsoft SQL Server with four separate databases; Redis stores Account and Project Membership authorization snapshots and WebSocket tickets; RabbitMQ acts as an event broker.
- Frontend: React/Vite runs in Backend mode and calls APIs through the Gateway. Flutter supplies the mobile interface using the system’s authenticated HTTP contracts.

2.4. Authorization, Trust Boundaries, and Controls

1. User Service authenticates login and issues RSA256-signed JWTs. The Gateway verifies signature, issuer, issue time, expiry, and UUID-form subject using the public key from User Service.
2. The Gateway reads auth:account:{accountId} from Redis. ACTIVE accounts receive a trusted X-User-Id header. Missing tokens result in 401, inactive Accounts in 403, and Redis or snapshot failures in 503.
3. Business services independently check roles or Project Membership. Task Service reads Redis membership snapshots and verifies Board/Column relationships through internal REST. The Gateway does not authorize Project roles.
4. Backend services use the user identity verified by the Gateway. Internal services should be restricted to a private network, with the Gateway as the only public API entry point and separate authentication for service-to-service calls.
5. Redis snapshots support rapid authorization decisions; the service-owned SQL database remains the source of truth. Member revocation creates synchronization jobs and version changes before Redis is updated.

03 Communication Diagrams — Two Main Use Cases

3.1. Use Case 1 — Login and Protected API Request

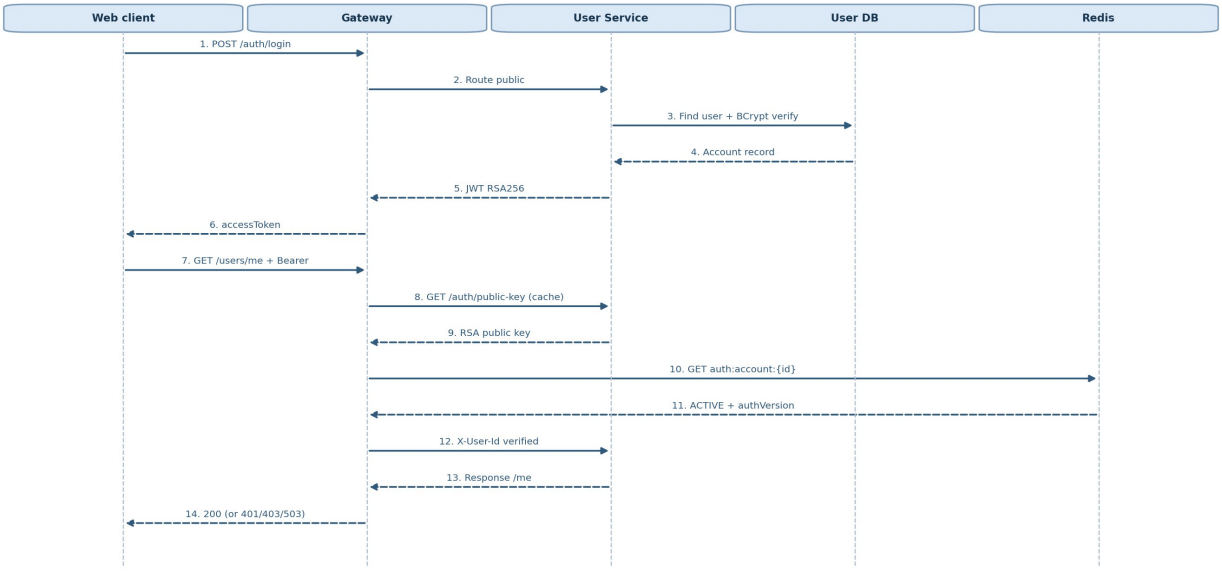


Figure 3.1.1. Communication sequence for public login and the protected GET /users/me request.

Table 3.1.1. Login and Gateway authentication steps

Steps	Mechanism / Relevant Classes
1–6	POST /api/v1/auth/login → AuthController → LoginUserService → UserRepository + BCryptPasswordAdapter → JwtTokenAdapter.issue; returns TokenResponse.
7–11	Client sends Bearer token; GatewayAuthenticationFilter obtains RSA public key (UserServicePublicKeyProvider) and RedisAccountSnapshotReader.read(accountId).
12–14	Gateway overwrites X-User-Id and routes to UserController.me (User Controller also parses/validates Bearer). Invalid tokens, inactive Accounts, and missing snapshots map to separate HTTP status codes.

Note: Login is public; the client uses the Bearer token for protected APIs afterward. The Gateway removes client-supplied identity headers, verifies the JWT with the public key, and checks the Account snapshot in Redis.

3.2. Use Case 2 — Assign Task → RabbitMQ → Notification



Figure 3.2.1. Task assignment with two Accounts: Account 1 assigns the Task; RabbitMQ distributes an event between services; Account 2 requests its notification through the Gateway.

Table 3.2.1. Processing phases for task assignment and notification creation

Phase	Behavior and Source Code Evidence
Request / Authorization	CardController.assignCard receives the identity verified by the Gateway and uses ProjectAuthorizationGuard to check Project Membership. The assignee needs ACTIVE Membership and a membershipVersion.
Write Transaction	AssignCardService locks membership assignment and updates Task/TaskAssignee. It emits card.assigned only when the new

Phase	Behavior and Source Code Evidence
	assignee differs from the prior assignee and the actor.
Outbox + RabbitMQ	TaskNotificationOutboxAdapter → OutboxEventWriter (MANDATORY transaction) → outbox_events. OutboxRelay publishes to exchange dev-platform.domain-events using routing key card.assigned.
Consumer + Inbox	NotificationRabbitConfiguration binds notification-service.task-events and a dead-letter queue; TaskEventListener → ProcessTaskNotificationService recordIfNew → saves notifications. Duplicate events are ignored.
Read Result	GET /api/v1/notifications/me?unreadOnly=false and PATCH /api/v1/notifications/{id}/read use the Gateway. The HTTP API does not wait for the Notification consumer to finish.

Event delivery is separate from the HTTP response: task assignment completes when its business transaction commits. Outbox Relay forwards events to RabbitMQ; the Notification consumer processes and persists them with eventual consistency. The Inbox record makes duplicate deliveries safe when the broker redelivers an event.

3.3. Supporting Flow — Invitation and Membership

The Owner sends POST /projects/{projectId}/invitations. Board Service checks the Owner and active Account through an internal user lookup, then stores a PENDING Invitation. The recipient sends POST /invitations/{id}/accept, which creates an ACTIVE Membership with a version and Redis snapshot synchronization job. Removing a Member returns a 202 operation handle; snapshot updates and assignment cleanup complete asynchronously. See ProjectApplicationService and the API controllers.

04 API Design

4.1. API Conventions and Routing

Terminology: this report calls a unit of work a Task. The codebase uses CardController, the cards table, and the /cards paths; these technical names are retained when reading code and calling APIs. Renaming endpoints to /tasks would require coordinated changes to the Gateway, Controllers, and Frontend.

The public base URL in the local configuration is http://localhost:8080/api/v1. Clients send Authorization: Bearer <token>, except for the two public endpoints (login and public-key). The Gateway injects X-User-Id. Board Service controllers typically return {data, meta}, whereas User/Task/Notification controllers return direct DTOs; the frontend must handle these response envelopes accordingly.

Table 4.1.1. API Gateway routing groups

Gateway Path Pattern	Target Service	Explanation
/api/v1/auth/**, /users/**, /admin/**	user-service	Public/protected Auth; SYSTEM_ADMIN required for administration.
/api/v1/projects/*/boards/*/cards/**; /projects/*/cards/**	task-service	Declared before the broad /projects/** wildcard route.
/api/v1/projects/**, /boards/**, /columns/**, /invitations/**	board-service	Project, Board, Invitation, and Membership.
/api/v1/notifications/**	notification-service	Personal Notification inbox.
/ws/**	task-service WebSocket	One-time WebSocket ticket; correct Origin and Board ID required.

4.2. Endpoint Matrix by Controller

Table 4.2.1. API endpoints grouped by service

Service	Method	API path	Purpose
User	POST	/auth/login	Login / JWT
User	GET	/auth/public-key	RSA public key
User	GET	/users/me	Current Account information
User	GET	/users, /users/profiles	Directory / Project-scoped profile
User	POST	/admin/accounts	Admin creates Account
User	PATCH	/admin/users/{id}/status	Admin changes Account status
User	GET/POST/PUT/DELETE	/admin/users/{id}/roles...	Manage system roles
Board	POST / GET	/projects	Create / list Projects
Board	GET / PATCH	/projects/{id}	View details / rename Project
Board	PATCH	/projects/{id}/status	Change Project status
Board	POST / GET	/projects/{id}/invitations	Invite / list Project Invitations
Board	GET	/invitations/me	My invitations
Board	POST	/invitations/{id}/accept|reject	Accept / reject Invitation
Board	GET / DELETE	/projects/{id}/members[/{userId}]	List / remove Members
Board	POST	/projects/{id}/transfer-ownership	Transfer ownership

Service	Method	API path	Purpose
Board	POST / GET	/projects/{id}/boards	Create / list Boards
Board	GET / PATCH	/boards/{boardId}	View / update Board
Board	POST / PUT	/boards/{id}/columns[/order]	Add / reorder Columns
Task	POST / GET	/projects/{p}/boards/{b}/cards	Create / list Tasks by Column
Task	GET	/projects/{p}/cards/{c}	Task details
Task	PATCH	/projects/{p}/cards/{c}/move	Move Task to Column/position
Task	PATCH	/projects/{p}/cards/{c}/assignee	Assign Task to Member
Task	GET / POST	/projects/{p}/cards/{c}/comments	Read / create Comment
Notification	GET	/notifications/me	My notifications
Notification	PATCH	/notifications/{id}/read	Mark notification as read
Gateway	POST	/realtime/boards/{id}/ws-ticket	Issue WebSocket ticket

The paths in the table omit the shared /api/v1 prefix. This list follows actual controller mappings, rather than automatically generated OpenAPI documentation.

4.3. Basic API Contract Examples

Login: POST /api/v1/auth/login

Content-Type: application/json

{"email":"owner@example.test","password":"<password>"}

200 OK

{"accessToken":"<jwt>","tokenType":"Bearer"}

Create Project: POST /api/v1/projects

Authorization: Bearer <jwt>

Content-Type: application/json

{"name":"Project Phase 1 Demo"}

201 Created, Location: /api/v1/projects/{projectId}

{"data":{"id":101,"name":"Project Phase 1 Demo","status":"ACTIVE","myRole":"OWNER"},"meta":{"...":"..."}}

Note: IDs, email addresses, JWTs, and JSON content are illustrative API examples. Use data generated by the running system during a live demo.

Assign Task: PATCH /api/v1/projects/{p}/cards/{c}/assignee

Authorization: Bearer <jwt>

Content-Type: application/json

{"assigneeId":"550e8400-e29b-41d4-a716-446655440000"}

200 OK → CardResponse (assignee updated)

Task pagination: GET /api/v1/projects/{p}/boards/{b}/cards?columnId=10&limit=5

200 OK

{"items":[{"id":42,"projectId":101,"boardId":7,"columnId":10,"title":"Review API"}],"nextCursor":"<opaque>"}

The backend uses a default limit of 5 and a maximum of 20; columnId is mandatory. Results are sorted by position DESC and id DESC. The cursor is scoped to Project/Board/Column. The UI loads more tasks per Column and deduplicates by cardId.

4.4. HTTP Status Codes and Authentication

Table 4.4.1. HTTP status codes and meanings

Situation	HTTP	Interpretation
Missing/incorrect Bearer, invalid JWT, invalid ticket	401	Unauthenticated request.
Account snapshot not ACTIVE or insufficient permissions	403	Access denied.
Project/resource does not exist or is intentionally hidden from non-members	404	Not found within the permitted scope.
Duplicate Invitation, invalid state transition, Membership conflict	409	Business-state conflict.
Backend dependency, snapshot, or Redis unavailable	503	Fail closed; permission cannot be verified.
Member removal continues to synchronize snapshots	202	Asynchronous operation in progress.

05 Database Design

5.1. Database per Service and Entity–Relationship Overview

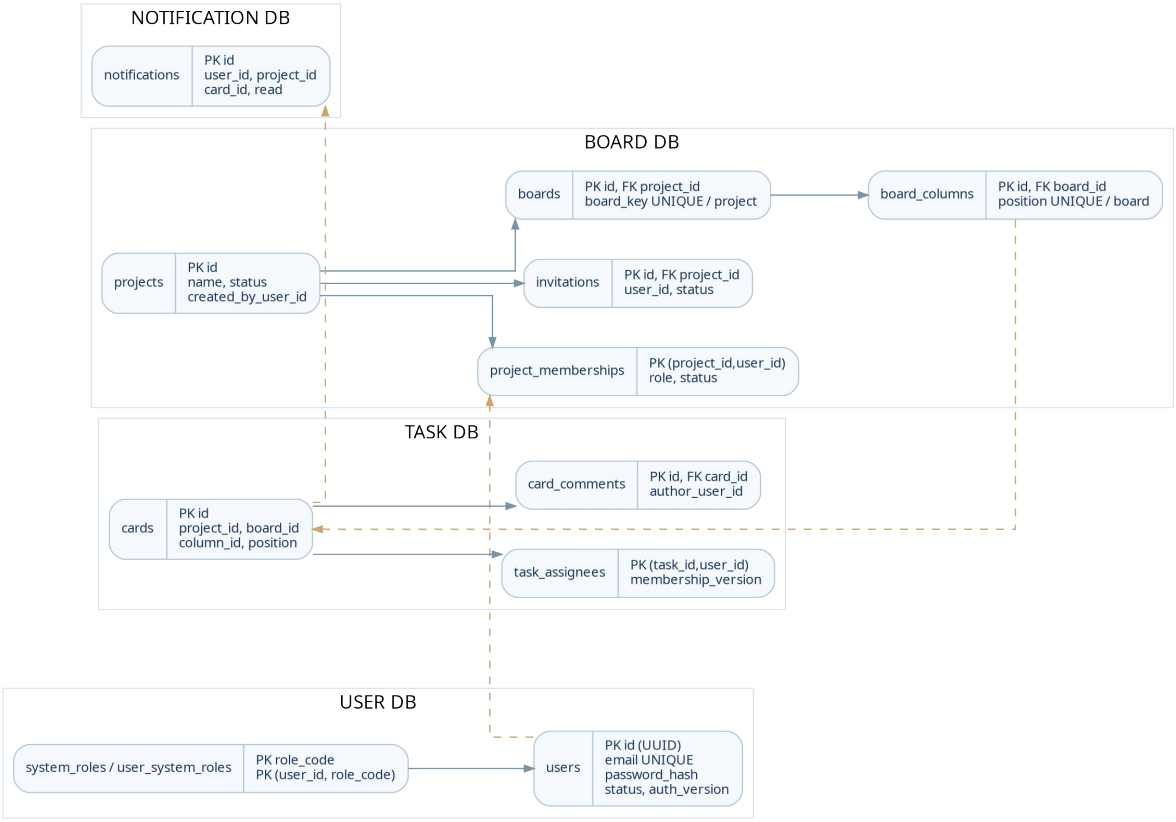


Figure 5.1.1. Logical data model. Database columns named card belong to the Task domain; there are no cross-database foreign keys between service-owned databases.

SQL Server contains four databases: kanban_user_db, kanban_board_db, kanban_task_db, and kanban_notification_db. Each service independently owns transactions and schema. The sql/local-sqlserver-bootstrap.sql script provides tables, constraints, and indexes for local development.

5.2. Service-Owned Database Tables

Table 5.2.1. Database tables and constraints by owning service

Database / Table	Primary Key and Representative Columns	Relations / Constraints
user_db.users	id UUID; email; password_hash; status; auth_version	UNIQUE email; never expose password_hash.
user_db.system_roles / user_system_roles	role_code primary key; (user_id, role_code) composite primary key	Local foreign keys to users/system_roles; SYSTEM_ADMIN is separate from Project roles.
user_db.account_authorization_snapshot_sync_jobs	job_id primary key; account_id; auth_version; status/retry	Unique per Account and authorization version; index supports scheduled retries.
board_db.projects	id BIGINT; name; status; created_by_user_id	Project root; status ACTIVE / COMPLETED / CANCELLED.

Database / Table	Primary Key and Representative Columns	Relations / Constraints
board_db.project_memberships	(project_id, user_id) primary key; role; status; joined_at	Local foreign key to Project; unique active Owner per Project (filtered index).
board_db.invitations	id primary key; project_id; user_id; invited_by_user_id; status	Unique PENDING invitation per (project_id, user_id), enforced by filtered index.
board_db.boards / board_columns	boards.id; project_id; board_key; columns.id; board_id; position	UNIQUE(project_id, board_key) and UNIQUE(board_id, position).
board_db.project_membership_versions	project_id / user_id; membership_version; authorization_version	Tracks Membership generation and authorization snapshots.
board_db.project_authorization_snapshot_sync_jobs	job_id; operation_id; project/user/version; retry/status	Durable synchronization job for Redis.
task_db.cards	id; project_id; board_id; column_id; title; position	Board/Project/Column IDs are scalar cross-database references.
task_db.task_assignees	(task_id, user_id); assigned_by; membership_version	Local foreign key to task_id; generation enables safe cleanup.
task_db.card_comments	id; card_id; author_user_id; content	Local foreign key to card_id in task_db.
task_db.membership_assignment_fences / cleanup_jobs	project_id; user_id; membership_version; status/retry	Fences and cleans assignments when Membership is revoked.
notification_db.notifications	id; user_id; project_id; board_id; card_id; read	No foreign keys to User/Task/Board databases; recipient belongs to an Account.
outbox_events / inbox_events	event_id primary key; (consumer_name, event_id) composite primary key	Stored in databases using messaging-support for retries and deduplication.

5.3. Index and Transaction Design

- Task pagination by Column: an index on Project ID, Board ID, Column ID, Task position, and Task ID supports loading additional tasks within each Column. Source index name: ix_cards_column_cursor.
- Notification listings: an index on recipient Account and read status supports user-specific queries. Technical database index name: idx_notifications_user_read.
- Outbox Relay: an index on delivery status, next retry time, and creation time helps the worker locate events awaiting distribution. Technical database index name: idx_outbox_due.
- User/Board snapshot jobs: uniqueness is enforced per entity and authorization version, with due-time indexes for scheduled retries.
- Project creation, Owner Membership, and snapshot synchronization job are committed in one Board Service transaction. Task assignment and its Outbox event share a Task Service transaction. Notification Inbox and Notification records share a Notification Service transaction.

5.4. Data Synchronization and Consistency

SQL databases are the source of truth. When an Account or Membership status changes, its service writes a durable snapshot synchronization job; a worker applies it to Redis. Gateway and Task Service rely on Redis for access decisions and fail closed when trusted data is missing. RabbitMQ uses at-least-once delivery; Inbox deduplication by (consumer_name, event_id) prevents duplicate notifications. No distributed transaction spans all four databases.

06 Main Demo Flow — Integrated Backend

6.1. End-to-End Business Flow Diagram



Figure 6.1.1. Chronological demo flow distinguishing Account 1 and Account 2, HTTP calls through the Gateway, and backend events distributed through RabbitMQ.

Main scenario: login → create Project → invite a member → accept Invitation → create Board → create Column → create Task → assign Task → write Outbox → RabbitMQ distributes event → create Notification → recipient views and marks it read → continue with Comments and move the Task between Columns. React runs in Backend mode and all HTTP APIs pass through the Gateway.

Table 6.1.1. Stages of the main demo scenario

Stage	Actor / API	Processing in the System
01. Login	Owner: POST /api/v1/auth/login	User Service verifies password and issues JWT; Gateway authenticates subsequent requests.
02. Create Project	Owner: POST /api/v1/projects	Board Service saves Project and Owner Membership in one transaction; synchronizes authorization snapshot.
03. Invite Member	Owner: POST /api/v1/projects/{p}/invitations	Board Service creates a PENDING Invitation linked to the Project and invited Account.
04. Accept Invitation	Member: POST /api/v1/invitations/{id}/accept	Board Service records ACTIVE Membership; authorization snapshot reflects new role.
05. Set Up Kanban	Owner: POST /api/v1/projects/{p}/boards; POST /api/v1/boards/{b}/columns	Board Service stores the Board and ordered Columns.
06. Create Task	Member: POST /api/v1/projects/{p}/boards/{b}/cards	Task Service validates Project permissions and Board/Column association, then saves the Task.
07. Assign Task	Member: PATCH /api/v1/projects/{p}/cards/{c}/assignee	Task Service updates assignee and writes card.assigned into Outbox within the transaction.
08. Publish Event	Outbox Relay → RabbitMQ domain-events	Event is published with a routing key and delivered to Notification queue.
09. Create Notification	Notification consumer receives card.assigned	Inbox checks event ID and saves Notification for the assignee.
10. Read Notification	Member: GET /api/v1/notifications/me	Notification Service queries database; client displays inbox and unread items.
11. Mark as Read	Member: PATCH /api/v1/notifications/{id}/read	Notification Service sets the notification read status.
12. Continue Collaboration	POST comments; PATCH card move	Task Service saves Comment/mention, moves Task between Columns, and publishes related realtime events.

Appendix A — Source Code Map by Topic

Table A.1.1. Source code references grouped by feature

Topic	File / Path in ZIP
Architecture / Decisions	README.md; docs/project/architecture-decisions.md; docs/project/06-integration.md
Gateway Routing / Auth	gateway/src/main/resources/application.yml; gateway/src/main/java/dev/platform/gateway/security/GatewayAuthenticationFilter.java
Gateway Redis / WebSocket	gateway/.../security/RedisAccountSnapshotReader.java; gateway/.../realtime/WebSocketTicketController.java
Auth / JWT	services/user-service/.../api/AuthController.java; .../application/service/LoginUserService.java; .../infrastructure/security/JwtTokenAdapter.java
Project / Invitation	services/board-service/.../api/controller/ProjectController.java; .../application/service/ProjectApplicationService.java
Board / Column	services/board-service/.../api/controller/BoardController.java; BoardColumnController.java; InternalBoardController.java
Task APIs / Assignment	services/task-service/.../api/CardController.java; .../application/service/AssignCardService.java; ProjectAuthorizationGuard.java
Task Event Outbox	services/task-service/.../infrastructure/messaging/TaskNotificationOutboxAdapter.java
RabbitMQ / Inbox / Relay	shared/messaging-support/.../OutboxEventWriter.java; OutboxRelay.java; InboxEventRecorder.java
Notification Consumer	services/notification-service/.../infrastructure/event/TaskEventListener.java; NotificationRabbitConfiguration.java
Notification Service	services/notification-service/.../application/service/ProcessTaskNotificationService.java; .../api/NotificationController.java
Database Schema	sql/local-sqlserver-bootstrap.sql; services/*/docs/database.md
React Backend Mode	web-demo/README.md; web-demo/src/features/; web-demo/src/shared/ (API client / Vite proxy)

Appendix B — Integration Checklist and Acceptance Criteria

Table B.1.1. Integration checklist and acceptance criteria

Category	Acceptance Test	Expected Observation
Login	Login issues JWT; Bearer token accesses /users/me; invalid token is rejected.	Correct user ID and HTTP status; Gateway validates token and Redis Account snapshot.
Project Authorization	Owner creates/invites; Member accepts; user outside Project cannot read its data.	Membership and snapshots accurately reflect permissions for each Account.
Kanban Data	Create Board/Column/Task, update assignee, move Task, and add Comment.	Data saved in appropriate service-owned database; cross-service reference IDs remain consistent.
RabbitMQ	Task Service writes Outbox and publishes card.assigned; Notification Service receives event.	Exchange and queue operate; event reaches Notification Service.
Event Deduplication	Resend the same event ID to the consumer.	Inbox deduplication produces only one Notification for the event.
Notifications	Member calls GET inbox and PATCH read.	Only current Account notifications are returned and read status is updated.
Realtime	Register Board WebSocket with valid ticket; perform a Task update.	Client receives change information or refreshes its data appropriately.
Secure Deployment	Internal services accept requests only across trusted network and identity boundaries.	Client calls public APIs only through the Gateway; internal credentials are not exposed in UI.

Scope of Interpretation

This report describes the complete Dev-Platform business flow and intended integrated architecture based on the source controllers, DTOs, entities, route configuration, repositories, and event handlers. Figures and expected outcomes illustrate the system in a configured integrated environment; they are not a record of end-to-end test execution or production data.

When presenting or demonstrating the project, run the steps in Section 6 and verify HTTP traces, databases, Redis, and RabbitMQ against Appendix B.